

RPG Python: lembrar, explicar e avançar

Uma retomada curta do que vocês já construíram — seguida de uma nova etapa de Programação Orientada a Objetos.

Vocês não estão começando do zero.

Hoje nós organizamos o que já existe e damos o próximo passo.

ROTA DE HOJE

1. lembrar a base
2. seguir o estado
3. explicar decisões
4. compor objetos
5. avançar para polimorfismo

A trajetória até aqui

A pausa não apagou o aprendizado. Ela apenas tornou necessário reativar as conexões.

ANTES DA PAUSA

- classes e objetos
- `__init__` e `self`
- métodos e atributos

DURANTE A PAUSA

- menos contato com o código
- detalhes ficaram distantes
- isso é esperado

DIA 1 — VOLTA

- revisão teórica
- leitura de exemplos
- vocabulário de POO

DIA 2 — PRÁTICA

- escrita de código
- testes manuais
- RPG funcionando

HOJE: transformar lembrança em compreensão que vocês conseguem explicar.

Vocês já construíram estas peças

Primeiro reconhecemos o terreno. Depois escolhemos o próximo desafio.

BASE DO RPG

- Classe Personagem
- objetos jogador e inimigos
- ataque, defesa e poção
- turnos e combate

POO JÁ PRESENTE

- estado dentro dos objetos
- métodos protegendo regras
- herança em Inimigo, Guerreiro e Mago
- composição com Item

Meta da retomada: apontar para o código e explicar por que cada peça existe.

O mapa do projeto

Arquivos diferentes podem ter responsabilidades diferentes.

PERSONAGEM.PY

- define o estado de Personagem
- guarda regras de dano e cura
- não controla o menu
- não contém o laço principal

MAIN.PY

- cria os objetos
- recebe escolhas
- organiza os turnos
- decide quando o jogo termina

Regra rápida: a classe conhece suas regras; o programa principal organiza o fluxo.

Classe é o molde; objeto é um estado real

A classe descreve possibilidades. Cada objeto guarda seus próprios valores.

personagem.py

```
class Personagem:
    def __init__(self, nome, vida):
        self.nome = nome
        self.vida = vida

jogador = Personagem("Thoric", 100)
goblin = Personagem("Goblin", 40)
```

LEIA ASSIM

- Personagem é a classe
- jogador e goblin são objetos
- cada objeto tem nome e vida próprios
- alterar goblin não altera jogador

Dois objetos da mesma classe podem estar em estados diferentes.

__init__ prepara o estado inicial

O construtor recebe os dados necessários e deixa o objeto pronto para participar do jogo.

personagem.py

```
def __init__(self, nome, vida, ataque, defesa):  
    self.nome = nome  
    self.vida = vida  
    self.vida_maxima = vida  
    self.ataque = ataque  
    self.defesa = defesa  
    self.defendendo = False
```

O QUE NASCE AQUI

- identidade: nome
- estado mutável: vida
- limite: vida_maxima
- capacidade: ataque e defesa
- condição temporária: defendendo

Se um atributo é necessário desde o nascimento do objeto, o construtor é o primeiro lugar para procurar.

self é o objeto que recebeu a chamada

Não traduza self como uma palavra solta. Descubra qual objeto aparece antes do ponto.

main.py

```
jogador.atacar(inimigo)
```

```
# dentro de atacar(self, alvo)
# self = jogador
# alvo = inimigo
```

PERGUNTAS

- quem executa o método?
- quem é self?
- quem chegou como alvo?
- qual objeto poderá mudar?

Leia como frase: jogador executa atacar usando inimigo como alvo.

Métodos alteram estado com regras

A mudança importante não é apenas uma conta: é a transição do objeto entre dois estados.

personagem.py

```
def receber_dano(self, quantidade):  
    dano_real = quantidade - self.defesa  
    if dano_real < 0:  
        dano_real = 0  
    self.vida = max(0, self.vida - dano_real)
```

ANTES E DEPOIS

- antes: vida = 40
- entrada: quantidade = 15
- defesa: 2
- dano real: 13
- depois: vida = 27

Compreender POO exige acompanhar o estado, não apenas ler a linha atual.

Objetos podem colaborar

Um objeto executa uma ação e envia uma mensagem para outro objeto.

personagem.py

```
def atacar(self, alvo):  
    dano = self.ataque  
    alvo.receber_dano(dano)
```

```
jogador.atacar(goblin)
```

COLABORAÇÃO

- jogador inicia a ação
- goblin recebe a mensagem
- goblin protege sua própria regra de dano
- o atacante não altera vida diretamente

Boa divisão: atacar escolhe o dano; receber_dano protege a vida.

Onde cada responsabilidade deve ficar?

Separar responsabilidades evita uma classe que tenta controlar o jogo inteiro.

DENTRO DO OBJETO

- como recebe dano
- como cura
- como defende
- quais limites deve respeitar

FORA DO OBJETO

- qual menu mostrar
- qual inimigo vem depois
- quando repetir o turno
- quando encerrar a partida

Pergunta-guia: esta regra pertence ao personagem ou ao fluxo do jogo?

Acompanhe um ataque completo

Siga a chamada até encontrar a linha que realmente altera o estado.

fluxo

```
jogador.atacar(goblin)
    |
    v
dano = jogador.ataque
    |
    v
goblin.receber_dano(dano)
    |
    v
goblin.vida = goblin.vida - dano_real
```

RASTRO

- a chamada começa no jogador
- o alvo é o goblin
- receber_dano calcula a redução
- somente o goblin perde vida

Não adivinhe: siga os objetos e escreva o estado antes e depois.

Defesa é um estado temporário

defender() prepara o objeto. receber_dano() consome esse estado no próximo ataque.

personagem.py

```
def defender(self):  
    self.defendendo = True  
  
def receber_dano(self, quantidade):  
    if self.defendendo:  
        quantidade = quantidade // 2  
        self.defendendo = False
```

CICLO

- False: estado normal
- defender(): muda para True
- próximo dano é reduzido
- receber_dano(): volta para False

A defesa vale uma vez porque o próprio método faz o reset.

A poção precisa respeitar limites

O método protege três regras antes de alterar vida e quantidade de poções.

personagem.py

```
def usar_pocao(self):  
    if self.pocoes == 0:  
        return  
    if self.vida == self.vida_maxima:  
        return  
    self.vida = min(self.vida + 25, self.vida_maxima)  
    self.pocoes -= 1
```

REGRAS

- não usar sem poções
- não usar com vida cheia
- não ultrapassar vida máxima
- diminuir somente após uso válido

O método mantém o objeto em um estado válido.

O turno traduz uma escolha em ação

O menu não calcula dano. Ele escolhe qual comportamento do objeto será chamado.

main.py

```
def turno_do_jogador(jogador, inimigo):  
    escolha = input("Escolha: ")  
  
    if escolha == "1":  
        jogador.atacar(inimigo)  
    elif escolha == "2":  
        jogador.defender()  
    elif escolha == "3":  
        jogador.usar_pocao()
```

RESPONSABILIDADE

- input retorna texto
- o if interpreta a escolha
- o objeto executa a regra
- o turno coordena, mas não calcula

Menu escolhe; método executa.

combate() controla repetição e encerramento

O laço continua apenas enquanto os dois participantes estão vivos.

main.py

```
def combate(jogador, inimigo):  
    while jogador.vida > 0 and inimigo.vida > 0:  
        turno_do_jogador(jogador, inimigo)  
  
        if inimigo.vida > 0:  
            inimigo.atacar(jogador)  
  
    return jogador.vida > 0
```

POR QUE FUNCIONA

- while expressa a condição
- o inimigo morto não age
- return informa vitória ou derrota
- main decide se continua

A condição protege o fluxo; os métodos protegem o estado.

Um jogador enfrenta vários objetos

A lista muda de inimigo, mas preserva o mesmo jogador e seu estado.

main.py

```
inimigos = [goblin, orc, troll]

for inimigo in inimigos:
    venceu = combate(jogador, inimigo)
    if not venceu:
        break
```

ESTADO PERSISTENTE

- jogador não é recriado
- vida continua entre batalhas
- poções continuam
- cada inimigo tem estado próprio
- break encerra após derrota

O mesmo objeto jogador atravessa toda a sequência.

A herança que vocês já experimentaram

Herança reaproveita uma base quando um novo tipo acrescenta comportamento real.

REAPROVEITA

- atributos de Personagem
- atacar e receber_dano
- defender e usar_pocao
- regras compartilhadas

ACRESCENTA

- Inimigo escolhe ação sozinho
- Guerreiro tem golpe_forte
- Mago possui mana
- super() prepara a parte herdada

Não avançaremos criando subclasses sem necessidade. Primeiro identificaremos responsabilidades.

A composição que vocês já tocaram

Composição aparece quando um objeto possui ou utiliza outros objetos.

É UM

- Guerreiro é um Personagem
- Mago é um Personagem
- Inimigo é um Personagem
- relação típica de herança

TEM UM

- Personagem tem inventário
- inventário contém itens
- item pode aplicar um efeito
- relação típica de composição

A nova etapa aprofundará o 'tem um' antes de voltar ao 'é um'.

Antes de avançar, consigam explicar

Sem programar ainda: conversem em dupla e apontem a resposta no código.

OBJETOS E ESTADO

- quem é self em jogador.atacar(goblin)?
- qual objeto perde vida?
- onde a defesa volta para False?
- por que a vida não fica negativa?

FLUXO E RESPONSABILIDADE

- por que input fica em main.py?
- por que o inimigo morto não ataca?
- por que o jogador não é recriado?
- quando usar composição?

Se a resposta ainda estiver confusa, voltamos ao rastro — não pulamos direto para mais código.

Um ciclo para resolver cada desafio

Use esta sequência para entender o problema antes de alterar o programa.

ANTES DO CÓDIGO

1. identifique a responsabilidade
2. leia um exemplo concreto
3. anote o estado inicial
4. preveja o resultado

COM O CÓDIGO

1. execute e observe
2. compare com a previsão
3. altere uma pequena regra
4. explique o novo resultado

Explicar > prever > executar > modificar > explicar novamente.

Como receber_dano() organiza as regras

Leia primeiro sem digitar. O objetivo é enxergar a sequência completa antes de executar.

personagem.py

```
def receber_dano(self, quantidade):  
    dano_real = quantidade - self.defesa  
    if self.defendendo:  
        dano_real = dano_real // 2  
        self.defendendo = False  
    dano_real = max(0, dano_real)  
    self.vida = max(0, self.vida - dano_real)
```

NARRAÇÃO

- recebe o ataque bruto
- aplica a defesa fixa
- aplica defesa temporária
- impede dano negativo
- impede vida negativa

Cada linha responde a uma regra do domínio do jogo.

Agora prevejam sem executar

Escrevam vida final, dano real e valor de defendendo para cada caso.

CASO A

- vida = 40
- defesa = 5
- defendendo = False
- receber_dano(17)
- qual é o estado final?

CASO B

- vida = 10
- defesa = 4
- defendendo = True
- receber_dano(30)
- qual é o estado final?

Não vale responder apenas o número: mostrem a ordem das operações.

Executem e comparem com a previsão

O teste serve para confrontar o modelo mental com o comportamento real.

teste_manual.py

```
heroi = Personagem("Herói", 40, 10, 5, 1)
print(heroi.vida, heroi.defendendo)

heroi.receber_dano(17)
print(heroi.vida, heroi.defendendo)

heroi.defender()
heroi.receber_dano(30)
print(heroi.vida, heroi.defendendo)
```

OBSERVEM

- a previsão acertou?
- qual linha explica a diferença?
- o estado temporário foi limpo?
- o objeto continuou válido?

Erro de previsão é informação: descubram qual regra não entrou no raciocínio.

Mudem uma regra pequena

Somente depois de compreender e testar a versão atual, alteramos o comportamento.

MUDANÇA

- defesa reduz 60% do dano
- o valor mínimo continua zero
- defendendo continua valendo uma vez
- não altere atacar()

CRITÉRIOS

- caso normal ainda funciona
- defesa é consumida
- vida nunca fica negativa
- vocês explicam a linha modificada

Mudança pequena, teste específico, explicação clara.

Explique o caminho do estado

Use o código e os valores do teste para reconstruir o que aconteceu.

MOSTRE NO CÓDIGO

- o estado inicial
- quem é self
- cada mudança de valor
- a linha da nova regra

CONFIRA

- onde o estado muda
- um possível caso limite
- se outra regra continuou funcionando
- o resultado com suas palavras

Se vocês conseguem ensinar a mudança, ela deixou de ser apenas uma cópia.

Quando falhar, sigam o estado

Debug não começa alterando linhas aleatórias. Começa reduzindo a dúvida.

SINTOMA

- vida não muda
- defesa vale para sempre
- inimigo morto ataca
- poção volta entre batalhas

PRIMEIRO TESTE

- mostre valores antes e depois
- confira o reset para False
- verifique a condição antes da ação
- procure onde o objeto é recriado

Leia o erro. Confira o objeto. Mostre o estado. Altere uma linha.

O RPG está crescendo

Mais funcionalidades dentro de Personagem fariam a classe conhecer detalhes demais.

PEDIDOS NOVOS

- itens de cura
- armas com bônus
- efeitos temporários
- inventário limitado
- itens consumíveis

PROBLEMA DE DESIGN

- Personagem ficará enorme
- muitos ifs dentro da classe
- cada mudança afetará tudo
- testes ficarão difíceis

Nosso próximo front: distribuir responsabilidades usando composição.

Antes de criar uma classe, faça três perguntas

Uma classe nova precisa representar uma responsabilidade compreensível.

PERGUNTE

1. qual informação ela guarda?
2. qual comportamento ela protege?
3. com quais objetos ela colabora?
4. por que isso não pertence à classe atual?

DECISÃO

- Personagem mantém vida e combate
- Item descreve algo utilizável
- Inventario organiza itens
- cada efeito sabe como ser aplicado

A arquitetura nasce de uma necessidade do programa, não de vontade de criar mais arquivos.

Personagem tem um inventário

O personagem não é um inventário. Ele possui e usa um objeto responsável por armazenar itens.

personagem.py

```
class Personagem:
    def __init__(self, nome, vida):
        self.nome = nome
        self.vida = vida
        self.inventario = Inventario()

    def pegar(self, item):
        self.inventario.adicionar(item)
```

RELAÇÃO

- Personagem tem Inventario
- Inventario guarda Item
- cada objeto mantém sua regra
- as classes colaboram por métodos

Composição conecta objetos sem transformar um no outro.

Item guarda dados e oferece um comportamento

Começamos com uma versão pequena. Novas regras serão adicionadas somente quando aparecerem.

item.py

```
class Item:
    def __init__(self, nome, valor):
        self.nome = nome
        self.valor = valor

    def usar(self, usuario, alvo):
        print(self.nome, "ainda não possui efeito")
```

ENTENDA

- nome identifica o item
- valor representa sua intensidade
- usar recebe os participantes
- a versão base ainda não decide o efeito

Primeiro definimos a mensagem comum: `usar(usuario, alvo)`.

Inventario protege a coleção

Em vez de espalhar operações de lista, damos um nome e uma responsabilidade ao conjunto.

inventario.py

```
class Inventario:
    def __init__(self):
        self.itens = []

    def adicionar(self, item):
        self.itens.append(item)

    def listar(self):
        for item in self.itens:
            print(item.nome)
```

POR QUE CLASSE?

- a coleção ganha um nome
- limites podem ser adicionados
- remoção fica centralizada
- Personagem não cuida da lista diretamente

Hoje é uma lista protegida; amanhã pode ter capacidade, busca e remoção.

Conecte os objetos primeiro

Implemente uma parte por vez e execute um teste pequeno depois de cada conexão.

PASSOS

1. criar item.py
2. criar inventario.py
3. instanciar Inventario em Personagem
4. adicionar pegar(item)
5. listar itens no main.py

TESTE MÍNIMO

- crie uma espada
- jogador pega a espada
- liste o inventário
- confirme o nome exibido
- explique qual objeto guarda a lista

Critério de pronto: o item aparece e a dupla explica o caminho do objeto.

Agora crie dois comportamentos

A estrutura já está conectada. Complete dois efeitos diferentes e compare como cada objeto reage.

ITEM DE CURA

- possui nome e valor
- aumenta vida do usuário
- respeita vida_maxima
- é removido após uso
- mostra o estado final

ITEM DE ATAQUE

- possui nome e valor
- causa dano no alvo
- usa receber_dano()
- não altera vida diretamente
- mostra o estado final

Antes de executar: prevejam qual objeto muda e qual método protege a regra.

A mesma mensagem, comportamentos diferentes

O jogo chama usar() sem precisar conhecer cada detalhe interno.

main.py

```
item = jogador.inventario.itens[escolha]
item.usar(jogador, inimigo)
```

```
# PocaDeCura.usar() cura o usuário
# Bomba.usar() causa dano no alvo
# ambos respondem à mensagem usar()
```

IDEIA CENTRAL

- a chamada é a mesma
- o objeto escolhido decide a versão
- main não precisa de um if para cada item
- novo item pode entrar depois

Primeiro percebam o padrão. Depois damos a ele o nome: polimorfismo.

O caminho das próximas aulas

Cada etapa nasce de uma limitação sentida no próprio RPG.

AGORA

- responsabilidade
- composição
- inventário e itens
- colaboração entre objetos
- testes de estado

DEPOIS

- encapsulamento mais forte
- polimorfismo em ações
- herança quando houver base real
- refatoração do combate
- testes automatizados

Não colecionaremos conceitos. Usaremos cada conceito para resolver um problema visível.

Mostrem, expliquem e escolham o próximo passo

A entrega vale pelo funcionamento e pela compreensão demonstrada.

DEMONSTRAÇÃO

- mostrar um personagem
- adicionar dois itens
- usar cada item
- mostrar mudança de estado
- mostrar um caso limite

SAÍDA

- onde composição apareceu?
- quem guarda a lista?
- quem aplica o efeito?
- por que usar() prepara polimorfismo?
- o que ainda está confuso?

Próxima aula: transformar os dois itens em uma família de comportamentos reutilizáveis.